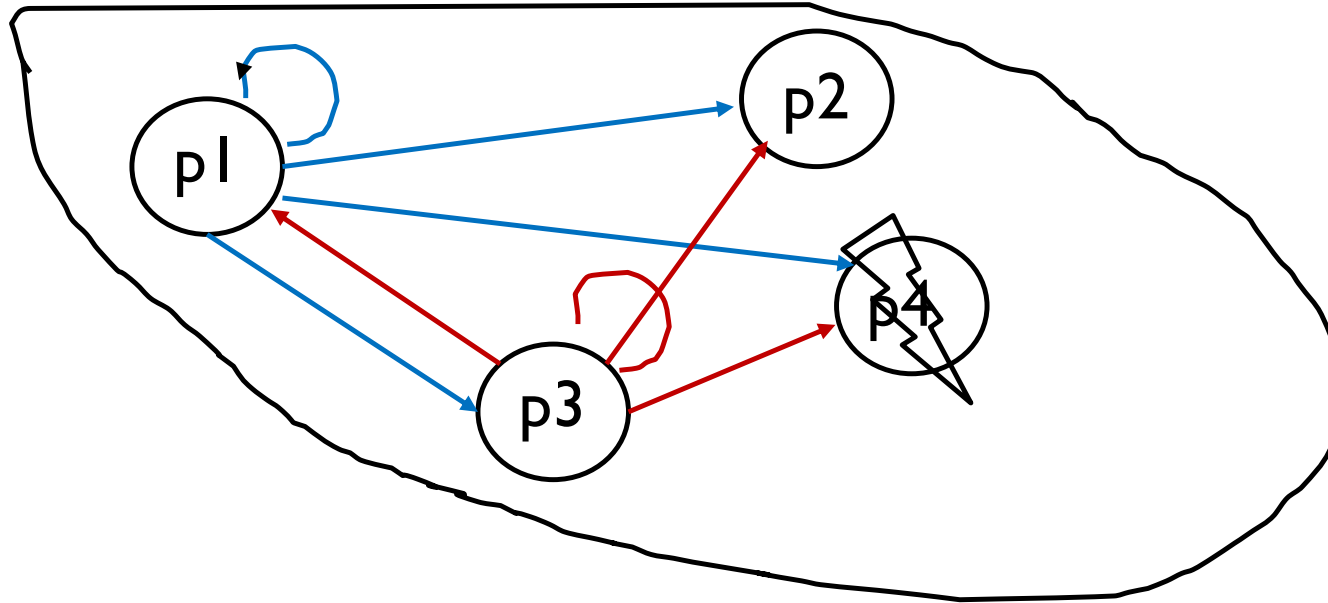

Group Communication Systems



- ❑ Provide the concept of a group
 - ☆ Can be dynamic – processes join and leave
- ❑ Multicast
- ❑ Ordering, e.g. FIFO, total,...
- ❑ Fault-tolerance

Group Concept

❑ Services

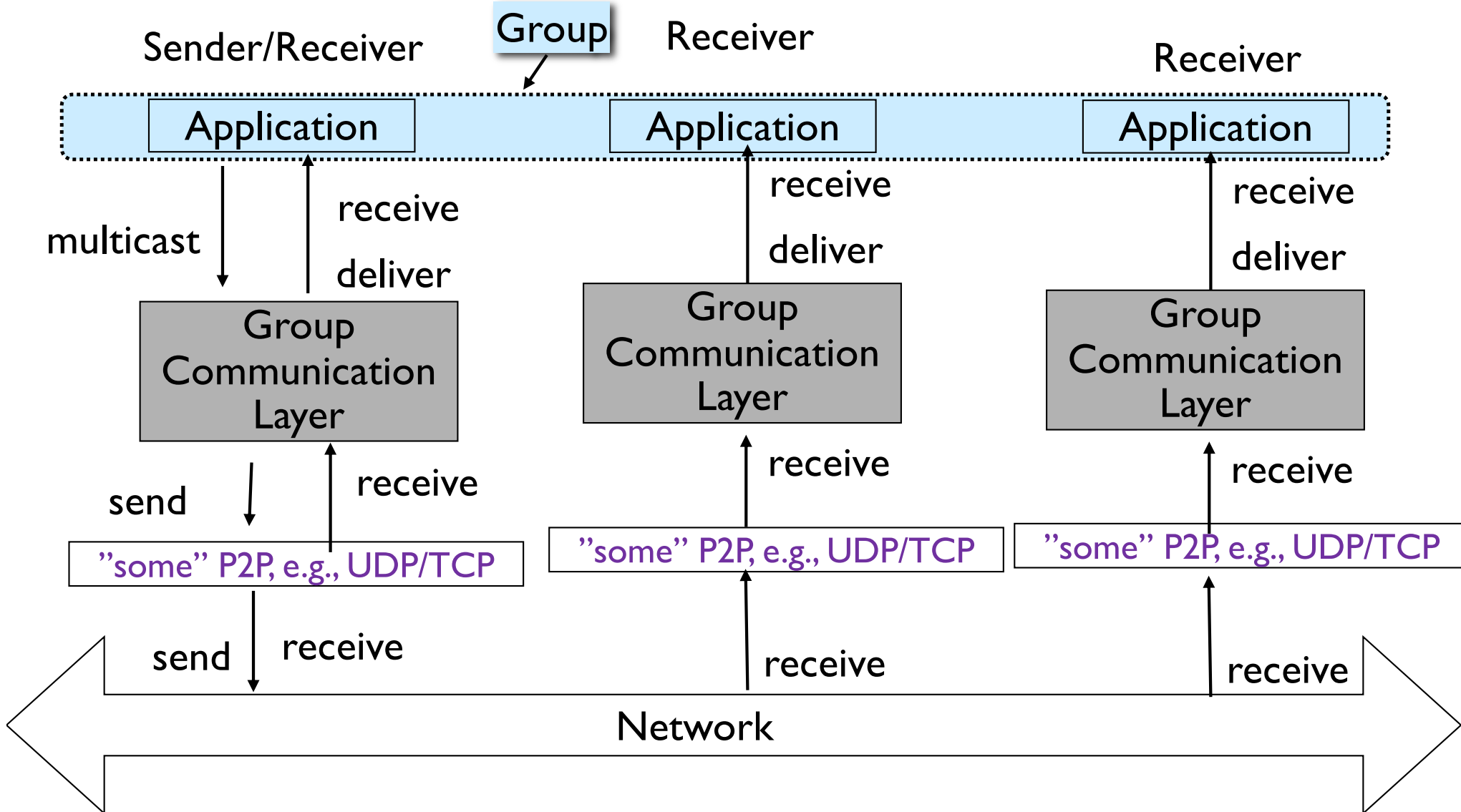
- ★ Group Maintenance

- ★ Multicast

Users of GCS

- ❑ Highly available servers (process replication)
- ❑ Video-Conferencing
- ❑ Chat system (Slack)
- ❑ Email groups?
- ❑ Data replication/caching
 - ☆ Document sharing
 - ☆ Shopping card

Architecture



What does the CL typically provide

- ☆ Connectionless vs. setting up a connection
- ☆ No *reliability* vs. *Reliability* despite message loss
 - e.g., implemented through ack and resending messages
- ☆ No *order* vs. *FIFO* delivery
 - Implemented through sequence numbers and delivery in order of sequence numbers
- ☆ TCP has connections, reliability and FIFO
- ☆ UDP is connectionless and guarantees no order or reliability
- ☆ One could imagine a protocol that implements a subset of these features – as they are, in principle, orthogonal

Services: Group Maintenance

☆ Group maintenance

- create / destroy groups
 - ▲ create(group-name)
 - ▲ destroy(group-name)
- processes can leave/join
 - ▲ join(group-name)
 - ▲ leave(group-name)
- automatic removal of failed nodes
 - ▲ (internal failure detection)
- application is informed whenever group composition changes
 - ▲ deliver group-change-event(new-config)

Services: Multicast

❑ Interface:

★ `multicast(group-name, message)`

★ `receive(group-name, sender, message)`

❑ Provides multicast options with different ordering guarantees

★ messages are delivered depending of the ordering level

- FIFO, causal, total

- *Multicast, FIFO-multicast, CO-multicast, TO-multicast*

❑ Provides multicasts with different reliability guarantees

★ The guarantee which messages are delivered depends on the reliability level chosen

Multicast Communication

❑ System model for now:

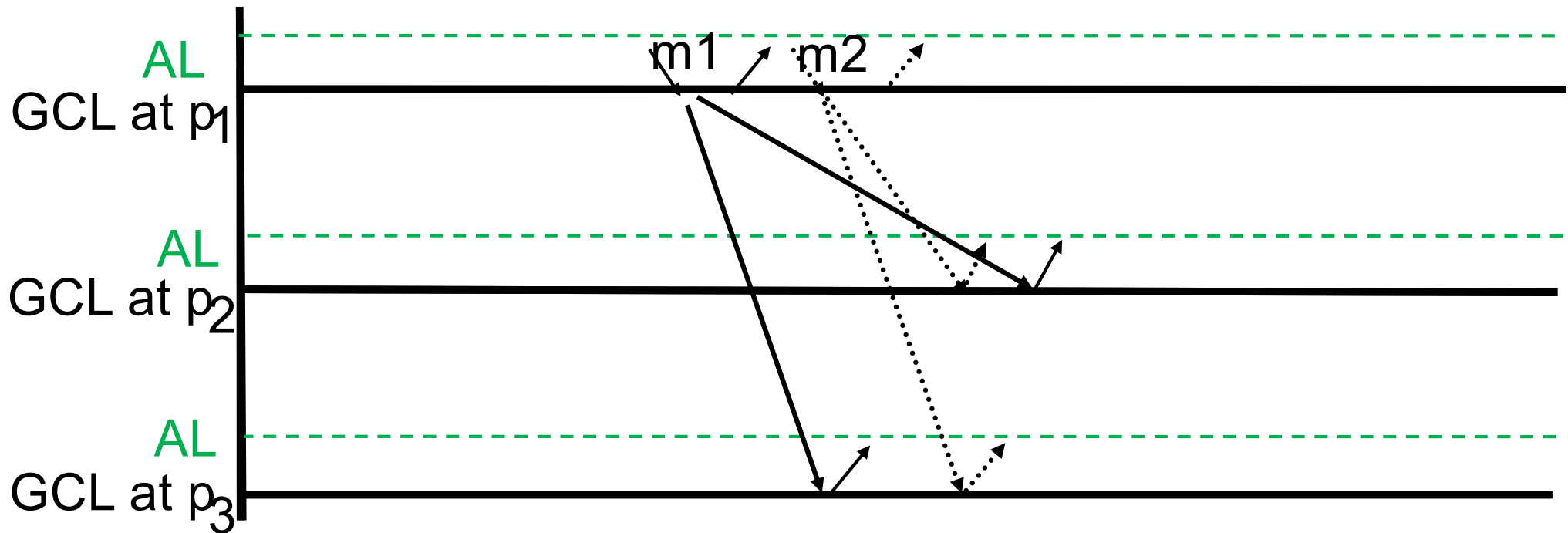
- ★ A group g consists of member processes $p_1, \dots, p_n$
- ★ no failures
- ★ There exists a communication layer that provides point-to-point send/receive primitives
 - reliable but no order

❑ Layers:

- ★ application layer: AL
- ★ gc layer: GCL
- ★ process: AL and GCL
 - from context it becomes clear which layer is referred to
- ★ communication layer: CL

No Order

- ❑ No order: multicast(g, m) —- *use when messages are unrelated*
- ❑ Messages might arrive in any order at the different nodes

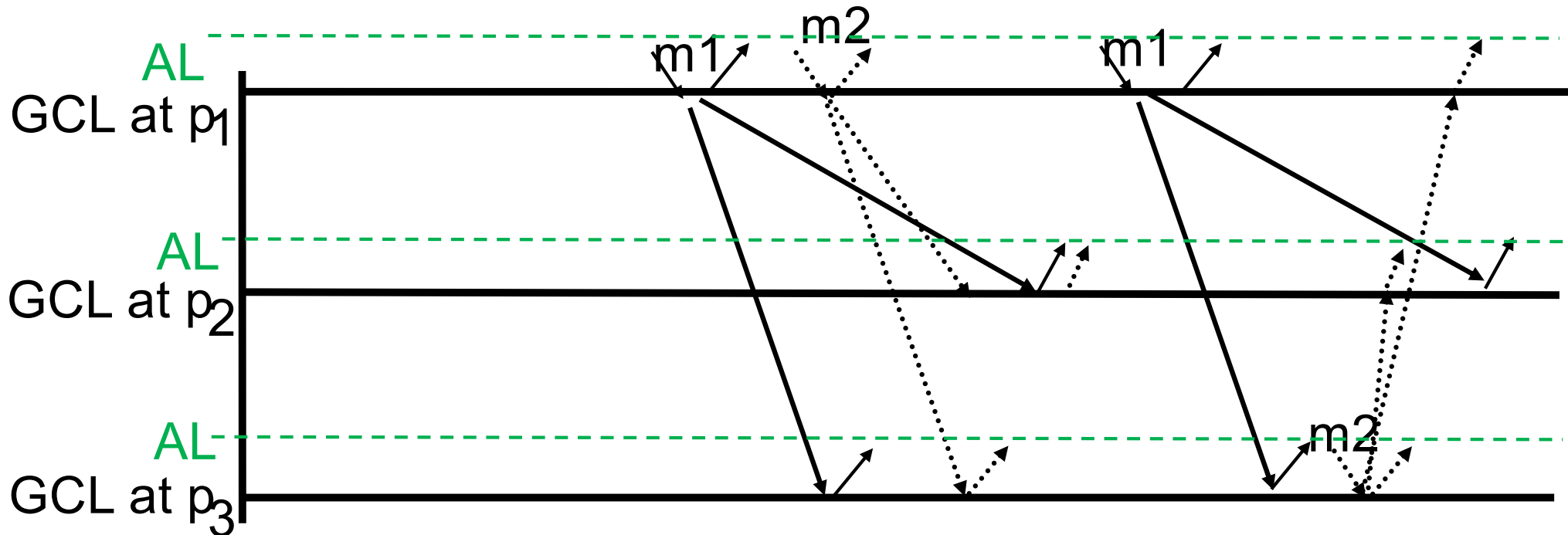


Simple no-order Multicast

- ❑ Assume CL provides
 - ★ `send(receiver, m)`
 - ★ `receive (sender, m)`
- ❑ At GCL of process p_i :
 - ★ Upon `multicast(g,m)` from AL
 - for all p_j of g : `send( $p_j$ ,  $\langle g,m \rangle$ )`
 - ★ Upon `receive( $p_j, \langle g,m \rangle$ )` from CL
 - `deliver( $g, p_j, m$ )`
- ❑ What happens with the sender?

FIFO

If the AL of process p_i has called $\text{FIFO-multicast}(g, m)$ and then $\text{FIFO-multicast}(g, m')$, then the GCL delivers m before m' at all member processes of g



Good for?

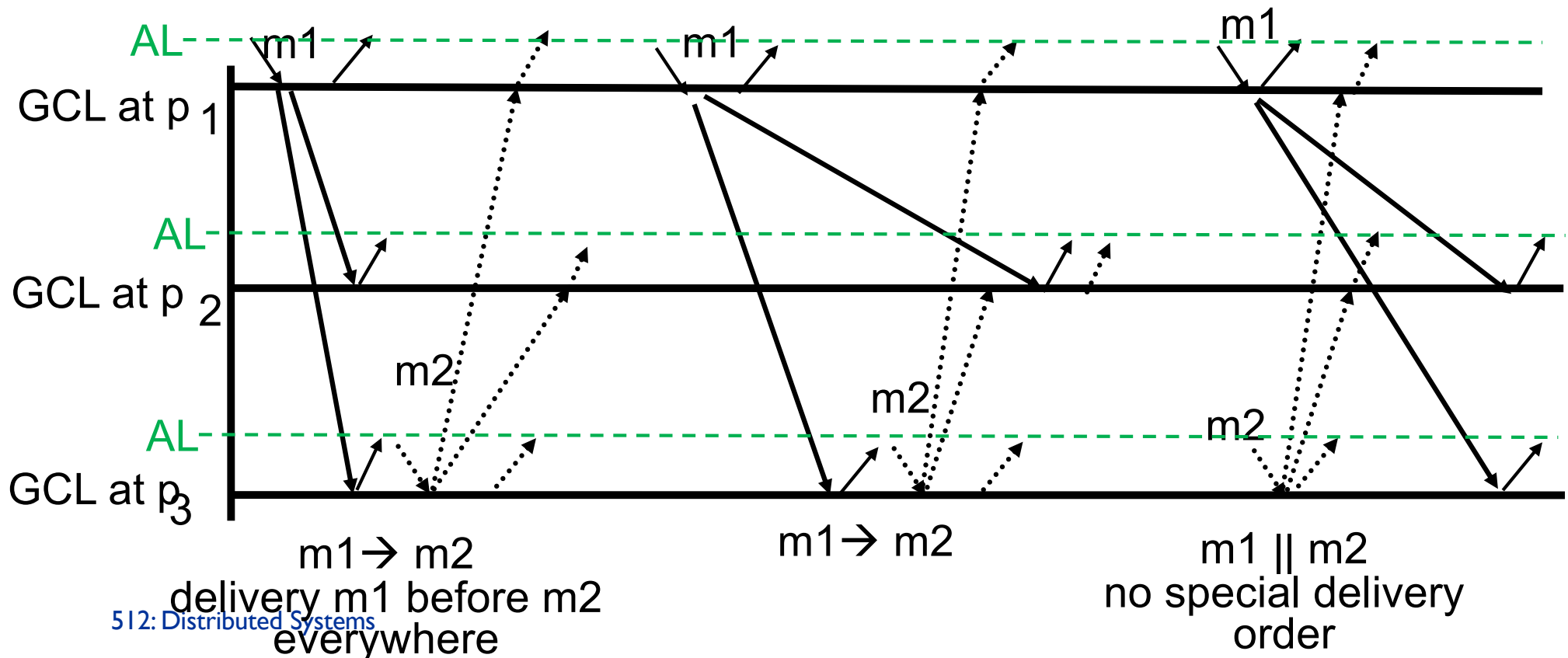
m_1 and m_2 from different
senders:
no special delivery order

FIFO Algorithm

CAUSAL

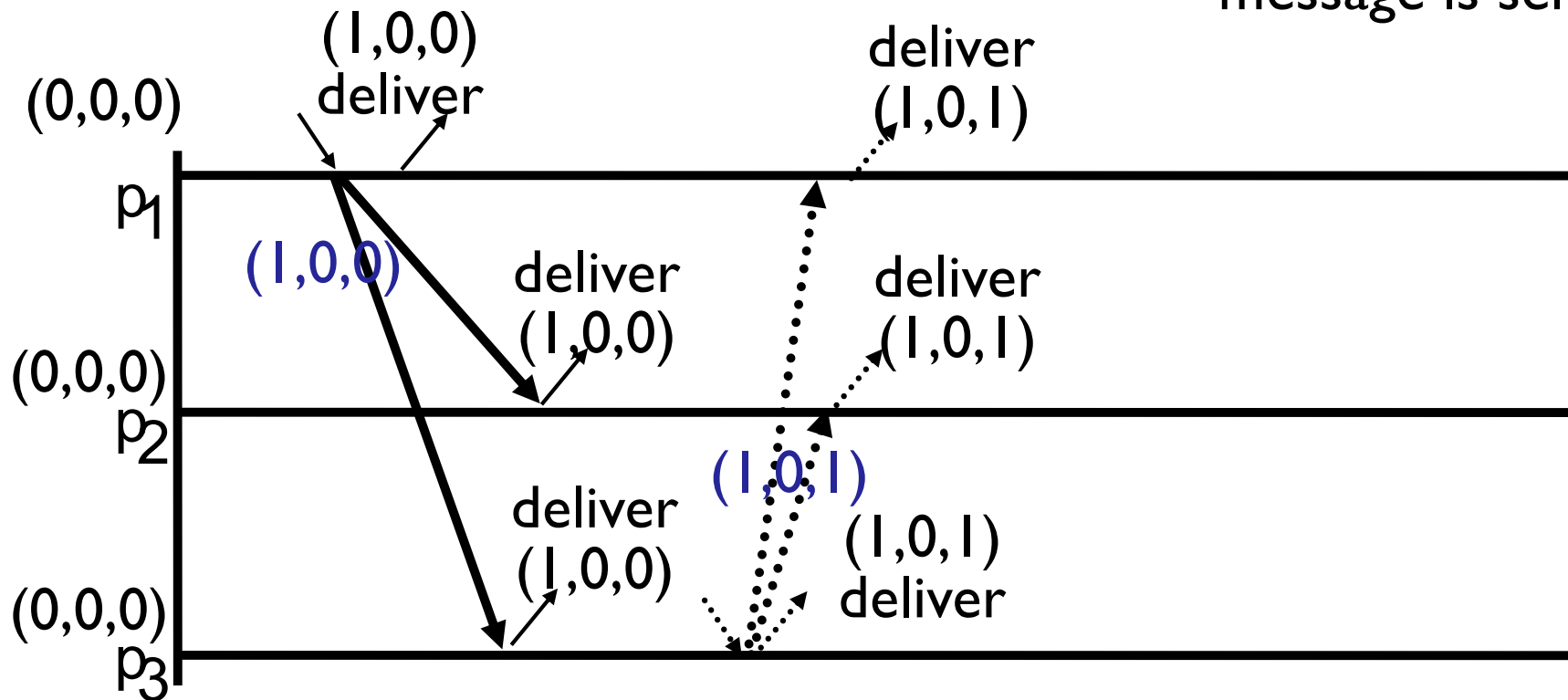
Good for?

- if $\text{CO-multicast}(g, m) \rightarrow \text{CO-multicast}(g, m')$, then the GCL delivers m before m' at all member processes of g .
 $\rightarrow$ is the happened before relation between messages
- Causal multicast guarantees FIFO but stronger
- the two multicast can be initiated by different processes
 - If the AL at p_i receives a message m from p_i and then multicasts m' , then $\text{CO-multicast}(g, m) \rightarrow \text{CO-multicast}(g, m')$,

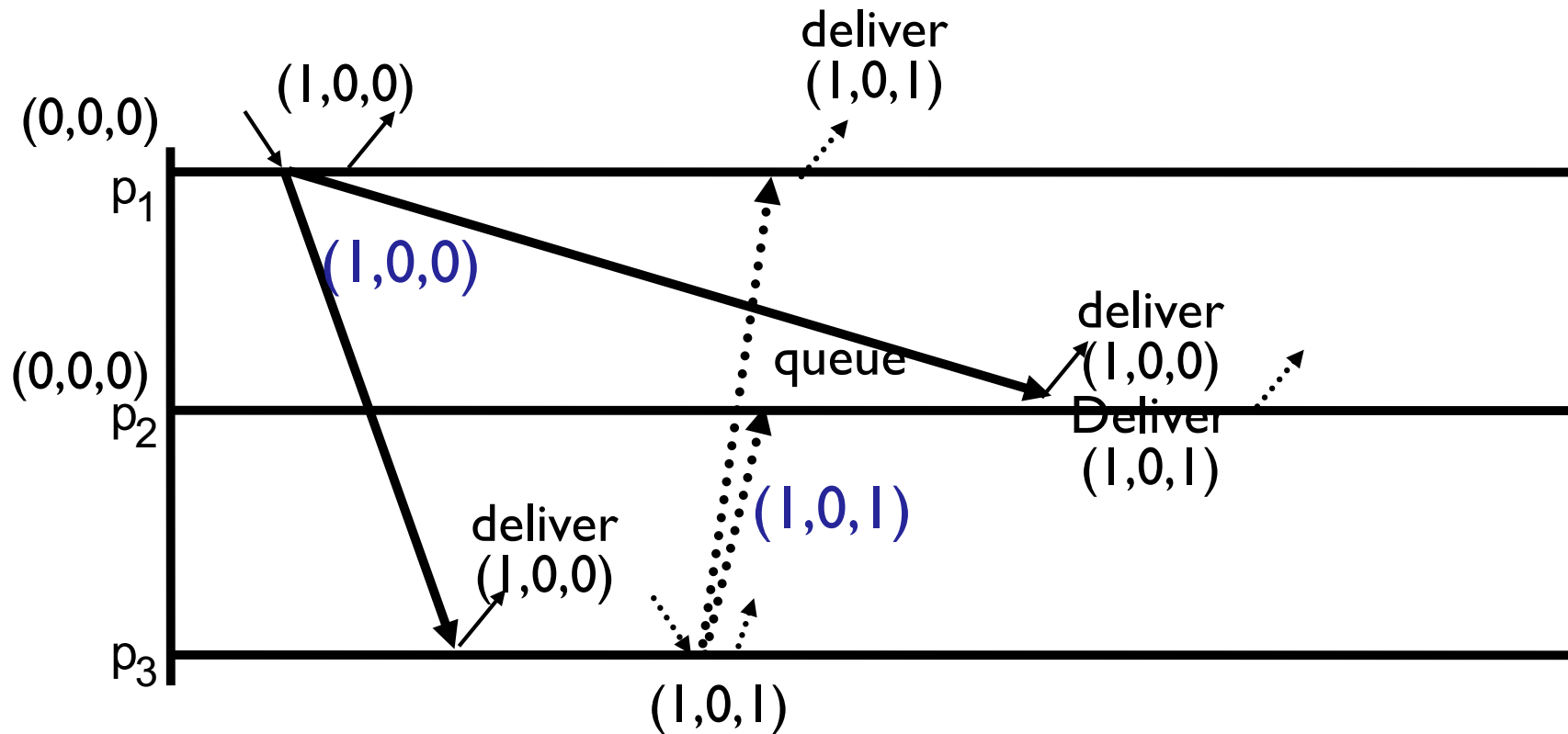


CO with vector clocks

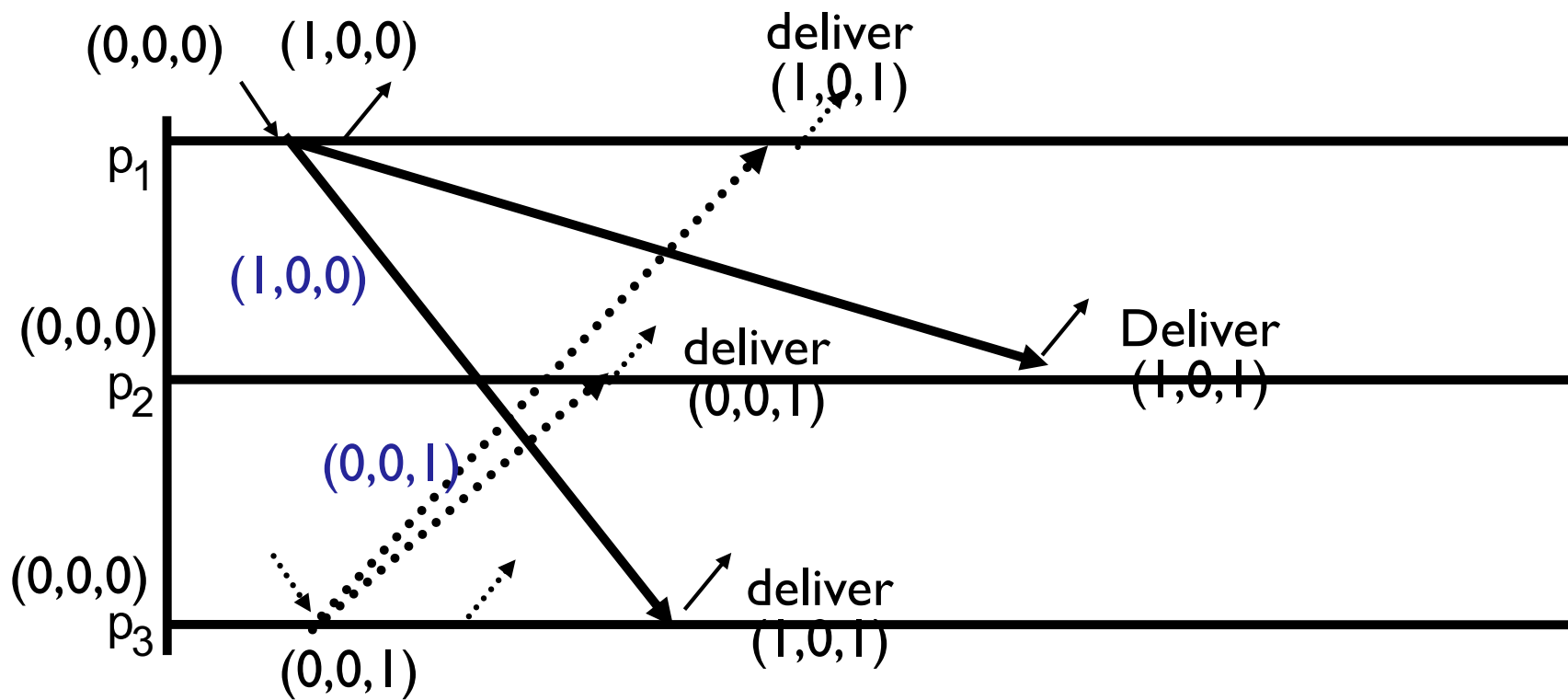
Vector clock only
increased when
message is sent



CO with vector clocks



CO with vector clocks



CO-Multicast algorithm

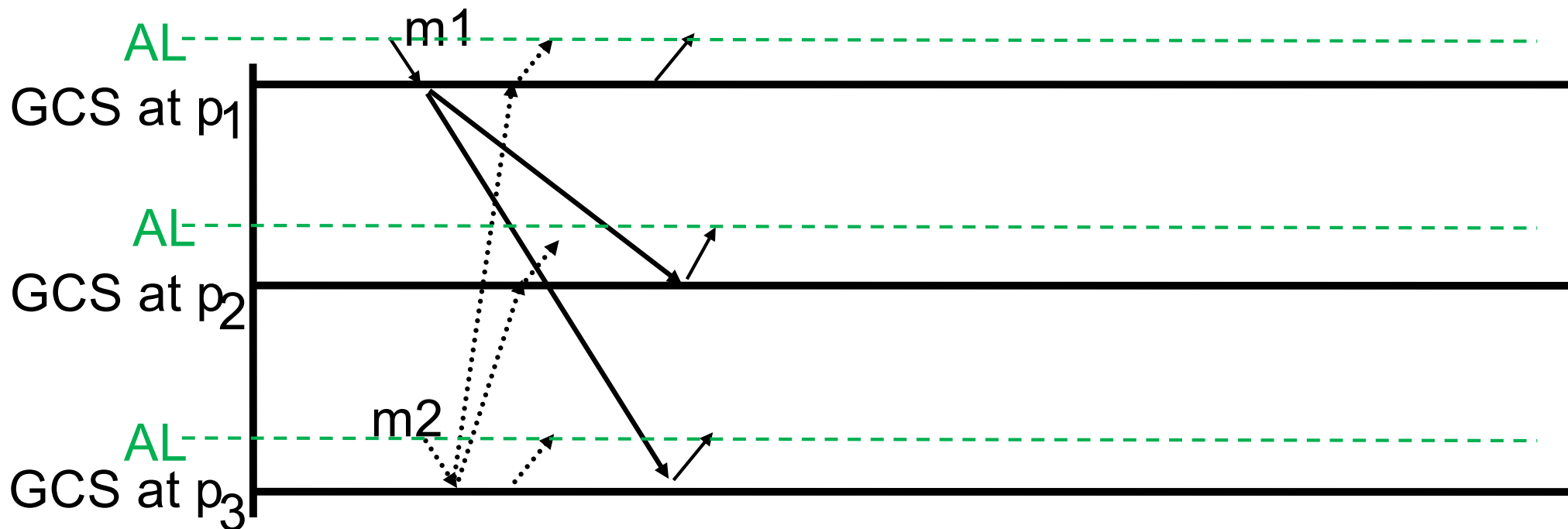
Causal Ordering using vector timestamps

- ❑ Assumptions:
 - ☆ existing point-to-point CL
 - ☆ Members of group g are processes p_i ($i = 1, 2, \dots, N$)
- ❑ CO-multicast/CO-deliver algorithm for GCS layer of process p_i ($i = 1, 2, \dots, N$)
- ❑ On initialization of group g
 - ☆ $V_i[j] := 0$ ($j = 1, 2, \dots, N$)
 - each group has own vector clock
- ❑ Upon CO-multicast(g, m) from AL
 - ☆ $V_i[i] := V_i[i] + 1$;
 - (NOTE: this is the only time the local timestamp is increased; other actions in the system do not increase the timestamp!)
 - ☆ for all p_j of g , i not equal j : send($p_j, \langle g, V_i, m \rangle$)
 - ☆ CO-Deliver (g, p_i, m)
- ❑ Upon receive($p_j, \langle g, V_j, m \rangle$) from p_j from CL
 - ☆ Place ($p_j, \langle V_j, m \rangle$) in queue;
 - ☆ Wait until $V_i[j] == V_j[j] + 1$ and $V_i[k] \leq V_j[k]$ ($k \neq j$)
 - ☆ Remove m from the queue and CO-deliver(g, p_j, m)
 - ☆ $V_i[j] := V_j[j] + 1$;

Comments on Performance

Example total order

for any two messages m and m' sent using TO-multicast, if the GCL delivers m before m' at any member process, then it delivers m before m' at all member processes



Good for?

any two messages must be delivered
in the same order everywhere

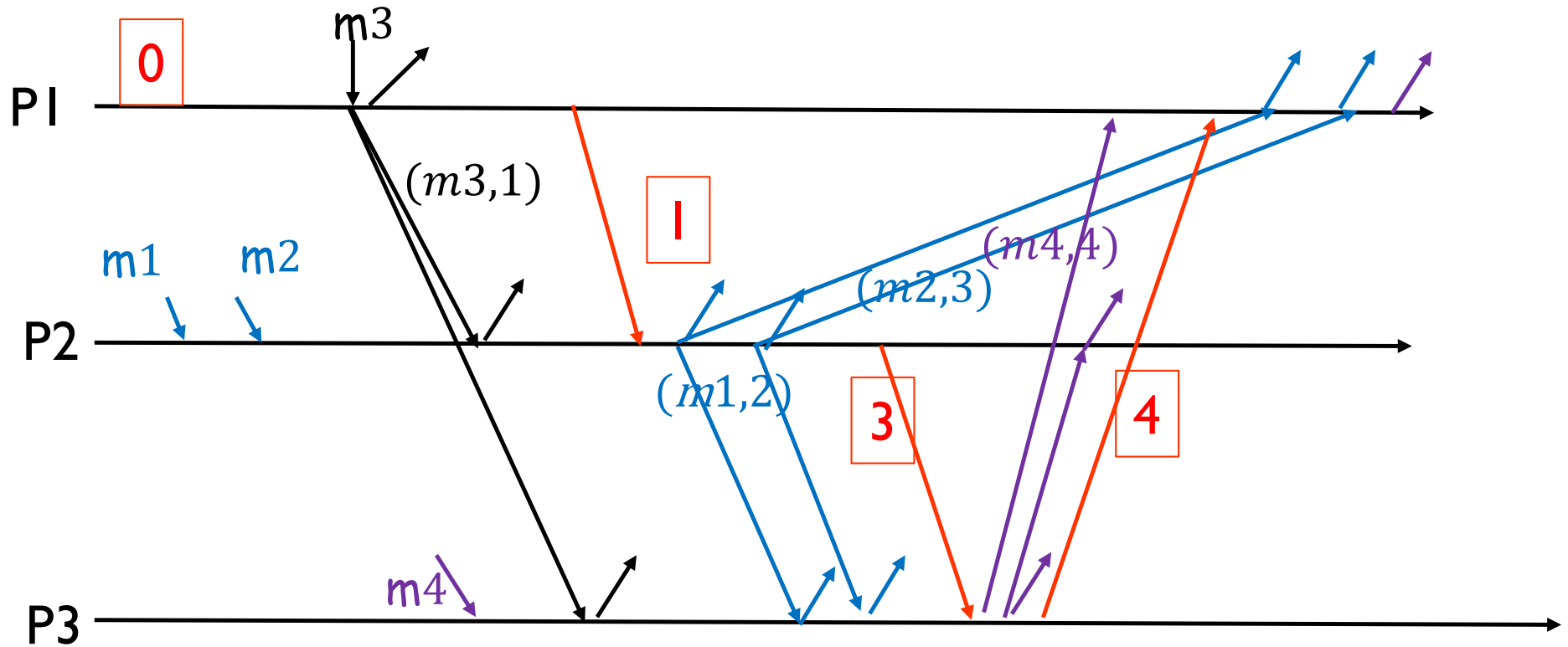
Total Order with coordinator process

- ❑ A specified process is the *coordinator* process
- ❑ A non-coordinator process sends a message *m* only to the coordinator (point-to-point)
- ❑ The coordinator FIFO multicasts its own messages and the messages it receives from others
- ❑ More formal algorithm voluntary as home exercise?

Total Order with sequencer process

- ❑ A specified process is the *sequencer* process
- ❑ Every process multicasts a message to all processes
 - ☆ (using FIFO or CO-multicast should the TO also be FIFO/causal)
- ❑ When a non-sequencer receives a message *m*, it queues the message
- ❑ When the sequencer receives a message it delivers
- ❑ Then it multicasts a “sequencer” message, indicating the order of the message in the stream of messages
 - ☆ (requires some message identifier)
- ❑ When a non-coordinator receives a “sequencer” message it delivers the corresponding message in sequence order
- ❑ 5.12: Distributed Systems More formal algorithm voluntary as home exercise?

Token based algorithm



Token protocol

- ❑ A token circulates among the member processes
- ❑ Whenever a process has the token it is allowed to multicast messages.
 - ★ Messages are tagged with sequence numbers
 - ★ Receivers deliver messages in order of their sequence numbers
- ❑ After a time period (e.g., no more local messages to multicast) the token is forwarded to the next process
 - ★ Token contains a field with the sequence number of last message multicast
 - ★ Next process uses this sequence number + 1 for its first message to be multicast

Token Protocol

❑ Assumptions

- ☆ One group g (separate data structures exist for each group)
- ☆ Underlying **simple multicast** (without any ordering guarantees)

❑ Messages

☆ Token t

- Seq: sequence number of the last message sent (high-water-mark)
 - ▲ Initialized to 0

☆ Message m

- Seq: sequence number of message

Token Protocol

Data structures at process GCL of p_i

- ★ Del_i : sequence number of last delivered message
 - Initialized to 0
- ★ NQ_i : queue of messages to be multicast (received from AL but not yet multicast)

Token Protocol 2

- Init (for group g)
 - ☆ $Del_i := 0$
 - ☆ $NQ_i := \text{empty set } s$
 - ☆ If p_i : create t , $t.seq = 0$
- Upon TO-multicast(g, m) from AL
 - ☆ Put (g, m) in NQ_i
- Upon receive token t by process p_{i-1} (or p_i after init)
 - ☆ While NQ_i not empty
 - Get next message of NQ_i
 - $t.seq = t.seq + 1$
 - $m.seq = t.seq$
 - Multicast(g, m) (or for all members, p2p message)
 - ☆ send token t to next process p_{i+1}

Token Protocol 3a

- ❑ Simple version:
- ❑ Upon receive(g, p_j, m) from p_j from simple multicast layer with $m.seq = s$
 - ★ Wait until $(Del_i + 1) = s$
 - ★ TO-deliver(g, p_j, m)
 - ★ Del_i++

Token Protocol 3b

- ❑ More detailed write up than previous page:
- ❑ Upon receive(g, p_j, m) from p_j from simple multicast layer with $m.seq=s$
 - ☆ Insert (p_j, m) into queue RQ_i sorted by sequence number
 - ☆ While RQ_i not empty
 - Look at first message (p_k, m')
 - If $(Del_i + 1) = m'.seq$
 - ▲ TO-deliver(g, p_k, m') and remove from RQ_i
 - ▲ Del_i++
 - Else break;

Analytical Performance Analysis

- ❑ Number of messages
- ❑ Number of hops/rounds
- ❑ others